Mechanic

Ongoing

The complete story of a tool that finds detection rules which quietly stopped working — including the day it discovered it had been wrong about its own core logic

Detection rule maintenance triage

Stages 1 and 2 complete · Externally validated · Stage 3 in design

Version 2.0 — a living document

Contents

Part I — The Problem

1. What a detection rule actually is
2. The silent death
3. Why nobody notices
4. The standard that exists and nobody uses
5. What else exists, and why it isn't enough

Part II — The Idea

6. Three signals
7. Fragility, and the Pyramid of Pain
8. Staleness, and what git remembers
9. Repair — the part nobody has done

Part III — The Machine

10. How the pieces fit together
11. discovery.py — finding the rules
12. loader.py — opening files that fight back
13. categories.py — the diagnosis book
14. churn.py — reading history without lying
15. ast_repr.py — turning text into shape
16. semantic_diff.py — real change vs cosmetic change
17. structural_detectors.py — the idea no wordlist can reach
18. protected_literals.py — strings that can't be renamed
19. fragility.py — combining the pieces
20. The borrowed parts

Part IV — What We Found

21. Eleven ways a rule repository breaks
22. Half of SigmaHQ
23. The real headline: Elastic and Splunk
24. The kappa climb: 0.412 to 0.800
25. The CIM discovery

Part V — The Turning Point

26. Looking for a ruler
27. The bug that eight rounds missed
28. The number that fell, and why that was good news
29. What the collapse proved about LLM labels

Part VI — Being Wrong Properly

30. The retraction
31. The citation that never existed
32. Everything else we got wrong
33. Limits, stated plainly

Part VII — Where It Stands

34. Current status, honestly
35. What is running right now
36. The contribution, in its final form

Part VIII — What Comes Next

37. Part 3: the priority ranking
38. Stage 3: repair and proof

39. Open questions

Appendices

A. Glossary
B. Prior art
C. Every number in one place
D. Datasets and sources
E. Timeline

Part I — The Problem

Before anything else: what a detection rule is, how one dies, why the death is invisible, and why a perfectly good industry standard for measuring this exists but almost nobody applies it.

1. What a detection rule actually is

Your computer writes down almost everything it does. A program started. A file was created. Someone logged in. A registry key changed. On a busy server that runs to millions of lines a day; across a company, billions.

Nobody reads that. It would be like being handed every phone call made in a city and asked to find the one where a crime was planned.

So security teams write **detection rules**. A rule is, at heart, an if-statement over the log:

```
if the program that started was named "mimikatz.exe"
then raise an alert
```

That's it. A filter with an alarm bolted to it. Mimikatz is a real tool that steals passwords out of Windows memory, so seeing it run on your network is worth knowing about.

Real rules are written in a structured format so they can be shared between companies and translated to different security products. The most common open format is **Sigma**:

```
title: Mimikatz Execution
logsource:
    product: windows
    category: process_creation
detection:
    selection:
        Image|endswith: '\mimikatz.exe'
    condition: selection
```

Read it top to bottom: name the rule, tell it to look at Windows process-creation logs, fire when a program whose path ends in `\mimikatz.exe` starts.

Thousands exist publicly. SigmaHQ, the main community repository, has over three thousand. Elastic ships around two thousand. Splunk ships around two thousand more. Companies take these as a starting point and adapt them.

Two things to hold onto

First, a rule is *just text*. Nothing enforces that it is correct, current, or useful. It is a file in a folder.

Second: that rule keys on a **filename**. Remember that. It's about to become the whole story.

2. The silent death

The attacker copies `mimikatz.exe` and renames it `svchost.exe`.

That's the entire attack on the rule. Two seconds. The program is byte-for-byte identical. It does the same thing. It steals the same passwords.

The rule looks for something ending in `\mimikatz.exe`. The log says `svchost.exe`. No match. The rule does not fire.

> **THE IMPORTANT PART**
>
> Nothing broke. No error appeared. No log line said "warning, rule bypassed." No dashboard turned red. The rule is still installed, still enabled, still syntactically perfect, still running against every event.
>
> It simply never matches anything, ever again.

Compare this to normal software. A bug crashes something, or returns a wrong answer a test catches, or a user complains. Failure announces itself.

A defeated detection rule does none of that. It goes quiet. And quiet is exactly what a working rule looks like on a day when nothing bad happened.

Silence is ambiguous. It means either "you are safe" or "you are blind," and there is no way to tell which from outside.

This isn't hypothetical

A 2024 paper from Fraunhofer FKIE and RWTH Aachen, presented at USENIX Security and awarded a Distinguished Artifact Award, took 292 Sigma rules covering Windows process creation and tried to evade them by hand.

Five techniques, none clever: insert characters the command line ignores, substitute a synonymous argument, drop unnecessary characters, reorder arguments, re-encode arguments.

Result: **38% fully evadable, another 7% partially.** Almost half of the most widely used open detection rules in the world, walked past by a person with an afternoon and no special skill.

3. Why nobody notices

Four reasons, stacking on top of each other.

The tests only check spelling

Rule repositories run automated checks. SigmaHQ has a good one. It verifies the YAML parses, required fields exist, ATT&CK tags are valid, the title isn't too long.

Every one of those is about *form*. None is about *meaning*. A rule keying on a filename an attacker abandoned in 2021 passes every check perfectly, because it is still beautifully-formed YAML.

The build goes green. Green means "this file is well-written." People read it as "this rule works."

Too many rules, too few people

A mature team has thousands of rules and a handful of engineers. Nobody has time to re-read three thousand rules asking, of each, "could an attacker still walk past this?" And even wanting to, there is no list saying which to check first.

Alerts are watched; rules are not

Every dashboard shows alert volume, alert types, response times. All of that measures *rules that fired*.

A dead rule produces no alerts. It contributes nothing to any dashboard. It is invisible by construction — monitoring sees things that happen, and a dead rule's defining property is that nothing happens.

Activity looks like maintenance

The subtle one, and it became the most important finding in the project.

Ask "when was this rule last updated?" and git answers. Often the answer is recent.

But *what* was updated? Frequently: a typo in the description. A new tag. A reformatted author field. A bulk change touching two thousand files because someone renamed a schema field.

None of that touches the detection logic. The rule's brain hasn't been examined in three years. Every measurement anyone has says it was touched last month.

> **THE FOUR FAILURES TOGETHER**
>
> Tests check spelling, not meaning. Too many rules to review by hand. Dashboards only see rules that fire. Cosmetic edits make dead rules look alive.
>
> Each is a gap in *observation*, not in skill. The engineers are good. They have no instrument pointed at this.

Someone measured it

SCYTHE built a regression suite for 57 of their mapped detection rules and ran it. Pass rate: 87.7%. Roughly one in eight had silently failed. Not edge cases — credential theft, lateral movement, privilege escalation. Techniques the team believed were covered.

Their conclusion is the sentence this project exists to answer: if 7 of 57 rules are broken, those are seven coverage gaps that no dashboard, no ATT&CK heatmap, and no AI-assisted alert triage tool will surface.

4. The standard that exists and nobody uses

Here is the part that surprised us most, discovered midway through the project.

A rigorous, industry-backed methodology for measuring exactly this already exists. It works. It has serious institutional support. And in public practice it is barely applied at all.

Summiting the Pyramid

In 2023, MITRE's Center for Threat-Informed Defense released **Summiting the Pyramid** (STP) — a methodology for scoring how robust a detection analytic is against adversary evasion, by examining the components inside it.

It was built with CrowdStrike, Fortinet, Fujitsu, IBM Security, Microsoft and Verizon Business. It has reached version 4.0. It leads the Center's 2026 roadmap.

Its reasoning is exactly the reasoning this project independently arrived at. Their worked example on a scheduled-task rule: the `schtasks.exe` filename rates weak because an adversary can rename it; the command-line arguments rate higher because changing them would require the source code; the analytic scores poorly overall because renaming the file defeats it.

SigmaHQ formally adopted it. The official Sigma specification defines an `stp` tag so any rule can carry its robustness score. MITRE's own repository asks the community directly to work through analytics using the methodology and send results to SigmaHQ so they become available to everyone.

What actually happened

Scanning every rule directory in SigmaHQ — 3,783 files — produced this:

Measure	Result
Files carrying an `stp` tag	**6** (0.16%)
Commits those tags arrived in	1
Authors	1
Methodology version tagged against	v1.0.0 (current is v4.0)
Commit's stated purpose	Exercising new pySigma parsing support for the tag format

Six tags. One commit. One person. Testing that the parser worked, against a methodology version three major releases out of date.

Separately, MITRE publishes its own scored-analytics dataset — analytics scored by their researchers directly, with permalinks to exact source-rule commits and free-text justifications. That file has roughly 90 rows.

So the total publicly available corpus of expert-scored detection analytics, worldwide, is on the order of a hundred rules. Against a public population in the tens of thousands.

THE FINDING, STATED CAREFULLY

The methodology is not the bottleneck. It is rigorous, industry-backed, formally adopted into the Sigma specification, and actively developed.

Manual application is the bottleneck. Scoring requires a human expert to read each analytic individually, and that does not scale past a few dozen rules.

One caveat that must always accompany this claim: absence of published tags is not proof of absence of use. Enterprise teams may score internally and never publish. The measurable claim is about *public, shared* scoring — which is precisely what MITRE asked for.

This reframes the entire project, and improves it. mechanic is not proposing a rival taxonomy. It is automating an established standard that exists, is endorsed, and is going unused for want of automation.

There is a further signal in the same direction: STP v4.0's changelog describes MITRE automating part of their own scoring work using AI and LLM assistance — on a different axis (telemetry confidence rather than analytic robustness), but the direction of travel is unmistakable. The organisation that invented the methodology has concluded manual application doesn't scale.

5. What else exists, and why it isn't enough

Several people have solved parts of this. Here is what each leaves undone.

Syntax and hygiene checking — solved

pySigma ships 80 validators covering identifiers, field names, taxonomy correctness, dangling conditions and much more. Thorough and good. Entirely about form. Not one asks whether a rule can be evaded.

Rule quality scoring — solved

A May 2026 paper scores Sigma rules across six dimensions: ATT&CK alignment, taxonomy correctness, false-positive risk, metadata completeness, redundancy, style. Fragility is not among the six, and it evaluated only the curated SigmaHQ corpus.

Evasion research — done by hand, on one rule type

The Fraunhofer work proved rules are evadable, but manually, on Windows process creation only. Its goal was building a machine-learning system to detect evasions after the fact — not finding fragile rules before they fail.

Rule generation — heavily solved

At least six systems read threat intelligence and write new Sigma rules: AutoSigma, LLMCloudHunter, IntelEX, SigmaGen, RulePilot, FALCON. A crowded field.

Rule repair — partly done, reactively

GRIDAI uses multiple LLM agents to generate and repair intrusion detection rules with hallucination validation. EvoSIEM hardens Sigma rules against evasion using LLMs. ARMS, a 2026 master's thesis, generates rule variants covering related ATT&CK techniques.

All three are **reactive**. You hand them a rule, or a new attack sample arrives, and they act. None answers the question an engineer actually has.

THE GAP

Robustness scoring, generation, evasion testing and repair have each been addressed. Scoring has a standard; it needs a human per rule.

What is missing is the layer that automates that scoring across a whole corpus and decides **which of the rules you already own need attention first**.

One more piece of evidence

Searching for existing work on detection rule triage, prioritisation and lifecycle management returned almost entirely **alert** triage — deciding which of the thousands of alerts a rule produces is worth investigating.

Almost nothing on triaging the *rules themselves*.

The industry is organised around "which of these ten thousand alerts matters?" Nobody is asking "which of these ten thousand rules is quietly dead?"

Part II — The Idea

If you can't read three thousand rules by hand, you need something that reads them for you and says where to look. That means deciding what "worth looking at" means.

6. Three signals

mechanic measures three things. They are deliberately separate, because they can disagree, and disagreement is information.

Signal	The question	How measured
Staleness	When did a human last change this rule's logic?	Git history
Fragility	How easily could an attacker walk past it?	The rule's structure and contents
Repairability	Can it be fixed — and does the fix work?	Rewriting, then testing against real logs

Combine the first two and you get a priority list: *these fifteen rules, out of three thousand, deserve this week's attention.* Add the third and the tool stops complaining and starts helping.

One design decision runs through everything: **output is "review this," never "this is bad."** The classifier is good, not infallible. A tool issuing confident verdicts it can't back trains people to ignore it.

7. Fragility, and the Pyramid of Pain

In 2013 David Bianco published the **Pyramid of Pain**: a ranking of the things you can detect an attacker by, ordered by how much it hurts them when you do.

At the bottom, things they change in seconds. At the top, things they can't change without giving up the attack.

Tier	What the rule keys on	Cost to evade
IOC	A file hash, an IP, a domain	Seconds
Artifact	A fixed path, a literal string	Minutes
Tool	One specific named program	Hours — use a different tool
TTP	Something unchangeable without losing the capability	Expensive — often abandon the technique

TTP means Tactics, Techniques and Procedures — the behaviour, not the tooling.

Complication one: some strings can't be renamed

The naive version says any literal string is fragile because the attacker can change it. That's wrong, and getting it wrong cost months. Consider:

```
\Software\Microsoft\Windows\CurrentVersion\Run
```

A Windows registry location. Programs listed there start at boot. It is a fixed string, and matching it is string matching.

But the attacker *cannot rename it*. Windows defines it. Use a different location and Windows won't start their program — which was the entire point. The string is fixed by the operating system, not by the attacker's choice.

These are **protected literals**:

> *A literal is protected when the attacker cannot rename it without losing the capability, because an operating system, protocol, or platform schema defines it.*

Qualifying examples: Windows autorun registry paths, well-known RPC named pipes like `svcctl` and `atsvc`, well-known security identifiers, Active Directory schema attributes, Linux persistence paths like `/etc/crontab` and `/etc/ld.so.preload`, Linux kernel syscall names.

Some are **context-gated**. A cloud audit action name is unrenameable in AWS CloudTrail logs because Amazon defines it; the same word in a Windows command line means nothing of the sort. So it counts as protected only when a cloud-audit marker appears in the same rule.

Complication two: strength that lives between fields

The heart of the project.

Here is the most durable rule in the SigmaHQ corpus, in plain English:

> *Fire when a program's filename on disk does not match the original filename recorded inside the program itself.*

Windows executables carry metadata including an `OriginalFileName` field, set at compile time. Renaming the file doesn't change it. If a file called `svchost.exe` reports internally that it is really `mimikatz.exe`, someone renamed something.

This catches *every renamed binary, forever*. No list of bad names. It doesn't care what the attacker renames things to. Close to unevadable.

Now look at the file. It contains ordinary filenames — `rundll32.exe`, `powershell.exe`. Nothing looks special.

Our first classifier read those, saw ordinary program names, and scored the rule **Tool**. The most durable rule in the corpus, marked weak, by a classifier working exactly as designed.

WHY IT FAILED

That rule's strength is not in any word it contains. It is in the *relationship* between two fields — they must match, and the alert fires when they don't.

You cannot see a relationship by counting words. No amount of extra vocabulary fixes this.

The resulting claim, in its final and deliberately narrow form:

> *Word-level analysis has a systematic blind spot: rules whose durability lives in a relationship between fields are consistently under-scored. Correcting the word lists fixes most problems — but structural detection is what recovers the specific class of rules no wordlist can reach, including the most durable rule in the corpus.*

Part VI explains why the original, bolder version had to be cut down.

Complication three, discovered late: AND is not OR

This one was invisible for the entire project until external validation exposed it, and it gets a full chapter in Part V. The short version, because it belongs here conceptually:

A rule usually checks several things at once. How should the rule's tier combine its parts?

mechanic took the **maximum** — the rule is as strong as its strongest component. That feels intuitive and it is wrong.

If a rule requires A **and** B, the attacker only has to defeat whichever is weaker. Both must match for the alert to fire; break either and it's silent. So an AND-linked rule is only as strong as its *weakest* link.

MITRE's methodology states this explicitly: AND combines by minimum, OR combines by maximum. And a Sigma `selection:` block combines its fields by implicit AND — so this is the common case, not an edge case.

8. Staleness, and what git remembers

Every rule lives in git, which records every change ever made. A free maintenance history — if read carefully. Read carelessly it gives nonsense, in two ways.

Trap one: bulk commits

Count commits naively and every SigmaHQ rule looks well-maintained. Then look at the commits:

Repository	Largest single commit	What it was
SigmaHQ	2,931 files	"Comply With v2 Spec Changes"
Splunk	2,068 files	"Initial commit of modified objects"
Elastic	1,064 files	"Add Supplemental Mitre Mappings"

One person ran a find-and-replace and every rule gained a commit. Not maintenance — a schema migration in maintenance's clothes.

The fix: exclude any commit touching 10% or more of the corpus. What survives is *organic* activity.

Any threshold invites "why 10%?" So mechanic reports at 5%, 10% and 20% simultaneously, and publishes every excluded commit with hash and subject line. Auditable rather than asserted.

Trap two: cosmetic edits

Even after filtering bulk commits: two rules, one where someone rewrote the detection logic and closed an evasion, one where someone fixed a spelling mistake. Both show a recent commit. Both count as maintained.

So mechanic classifies every change into four buckets:

Bucket	What changed	Verdict
Behavioral	`detection:` or `logsource:`	Real maintenance
Semantic metadata	ATT&CK tags, status, severity	Meaning shifted, logic didn't
Cosmetic	Title, description, author, references	Nothing happened
Creation	The commit creating the file	Its own category

`logsource` counts as behavioral because changing `product: windows` to `product: linux` changes what the rule means entirely, despite sitting outside the detection block.

Semantic metadata gets its own bucket deliberately. An examiner will ask which bucket a tag change lands in; "its own, and here's why" beats an arbitrary assignment.

9. Repair — the part nobody has done

What a repair looks like

The mimikatz rule keys on a filename — the weakest possible thing.

Old: the file is named `mimikatz.exe`.

New: look at `OriginalFileName`, the identity compiled into the file that renaming doesn't touch; and look for the command mimikatz must be given to do its job, `sekurlsa::logonpasswords`. Change that and it doesn't steal passwords, which was the point.

Hand-tested against three evasions, that repair caught all three.

Three honest outcomes

A hand-run experiment on five rules gave: two clean repairs, one partial, one that broke the taxonomy and taught us about protected literals, one impossible. Hence:

Outcome	Meaning	Example
Logic repair	Rewrite to key on something durable	Filename → internal metadata + required arguments
Telemetry repair	No rewrite can fix it; the logs lack the data	PowerShell obfuscation needs script-block logging
Retire	Nothing durable underneath	A rule matching the literal `'quasar client'`

The middle one is valuable and nobody outputs it. Telling an engineer "*stop editing this YAML, your logs don't*

contain what you need" saves a day of futile work.

Proving the repair worked

Anyone can claim improvement. Proof separates this from a demo. A repair passes only if **all four** hold:

1. The new rule catches the evasions the old one missed
2. The new rule still catches the original attack
3. The new rule doesn't fire on normal, harmless activity
4. The repair preserves intent — same technique, same data source

Condition three matters more than it looks. A rule alerting on everything technically catches every evasion, and is worse than the original because now a human wastes time daily. Widening a rule until it always fires is the easiest way to fake success, and the benign baseline is what stops it.

How is a repair different from writing a new rule?

Three ways; the second matters most.

Input differs. Generation starts from a threat report and produces something from nothing. Repair starts from an existing rule plus evidence it's weak.

Repair has a baseline. A new rule has nothing to compare against — is it good? Nobody knows without months of production data. A repaired rule has a before and after, testable against identical inputs, today, on a laptop. Generation structurally cannot do this.

Trust differs. A new rule is an unknown entering production. A repair is a three-line diff on a rule already running, with evidence attached.

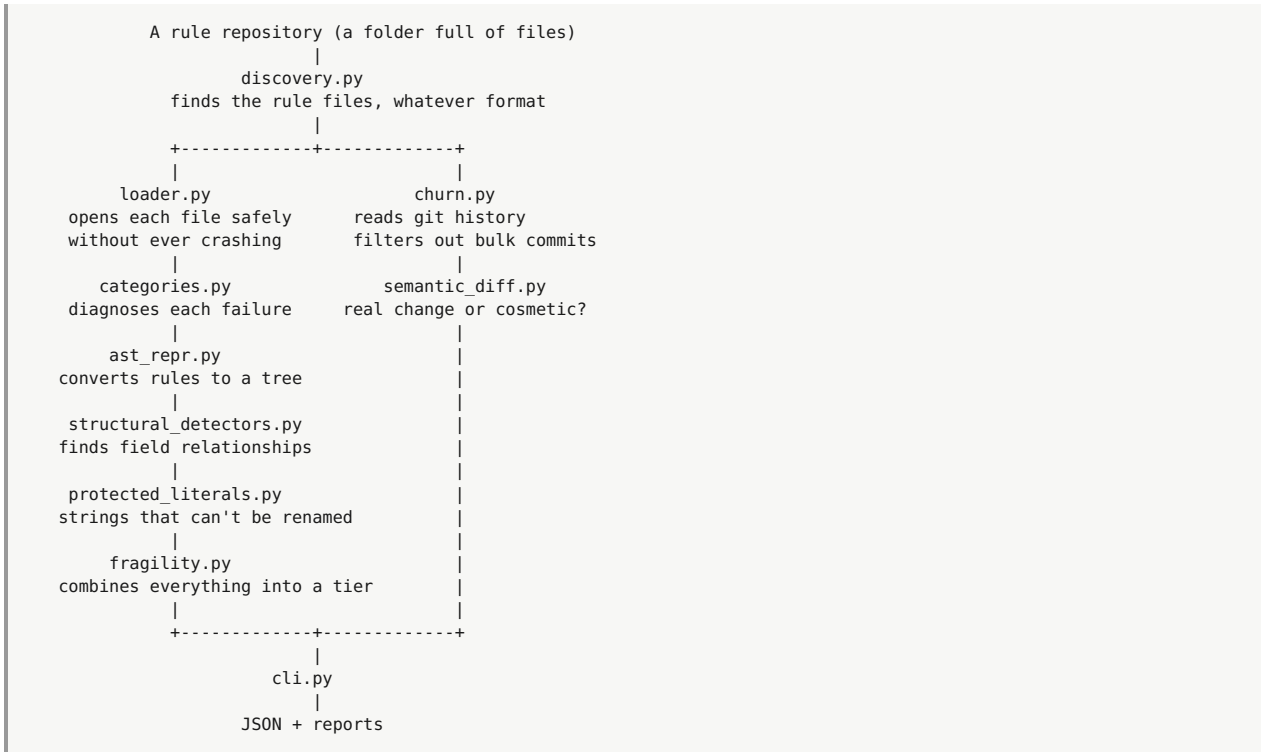
And the boundary: if fixing a rule means replacing every line, that's generation in disguise. So — a repair keeps the rule's *intent* and changes *what it keys on*. If intent must change, it's a new rule, and mechanic should say so.

> **IN ONE SENTENCE**
>
> A new rule is a guess. A repair is a change you can measure — because the old version gives you something to measure against.

Part III — The Machine

Every component, what it does, why it exists. Several were built only because something else broke first.

10. How the pieces fit together



Two independent chains. The left is about what a rule *is*; the right about what has *happened* to it. They meet only at the end.

That separation is deliberate. The staleness chain works perfectly even when the fragility classifier is wrong — which, twice now, it has been.

11. discovery.py — finding the rules

The least glamorous component. It walks a directory and returns rule file paths.

It matters because it is **format-agnostic**. Sigma writes YAML. Elastic writes TOML with queries in EQL or KQL. Splunk writes YAML with searches in SPL. Three formats, three query languages.

Because discovery is pluggable, staleness works on *any* rule repository — git doesn't care what's inside the file. Only deeper analysis needs to understand contents, and that currently requires Sigma.

It also handles `--subdir` scoping. SigmaHQ's repository contains rules, documentation, tests and schemas; count commits across all of it and a documentation overhaul looks like rule maintenance.

12. loader.py — opening files that fight back

This exists because of a genuinely surprising Phase 0 discovery.

The standard tool, `sigma check`, **crashes on four out of five real third-party rule repositories.**

On joesecurity's set — 119 files — every file fails and zero rules reach validation. You get a Python traceback and nothing else.

The cause isn't one bug. It's the absence of defensive coding anywhere in the ingest path. pySigma assumes well-formed rules. Real repositories contain rules written by hand, generated by machines, written in 2019 against a

spec that no longer exists, or committed by someone who never ran a linter.

Why one layer of protection isn't enough

The obvious fix is a try/except around loading. Insufficient — and finding out why shaped the design.

Two separate places a rule can kill the run:

During parsing. The YAML is malformed, or the constructor rejects something. Wrapping the load catches these.

During validation. The rule parsed *perfectly*. pySigma accepted it. Then a validator touches it and dies — for instance calling a regex against a null `references:` entry.

The second is invisible to any protection around loading, because loading succeeded. So mechanic uses **two layers**:

```
for each file:
    try:
        parse the YAML
        construct the rule object
    except:  ---> record structured failure, continue

    for each validator:
        try:
            run this validator on this rule
        except:  ---> record structured failure, continue
```

Failures become data, not tracebacks

```
sigma check:
    Traceback (most recent call last):
      File "sigma/rule.py", line 412, in from_dict
        UUID(rule_id)
    AttributeError: 'int' object has no attribute 'replace'

mechanic scan:
    rules/win_susp_process.yml
      category:  bare_int_id
      stage:     rule_construct
      hint:      the id field is a plain number; YAML reads it as an
                 integer, and pySigma expects a UUID string.
                 Quote it, or replace it with a real UUID.
```

Same failure. One says the library crashed. The other says how to fix your file.

13. categories.py — the diagnosis book

If loader.py is the mechanic, this is the manual of symptoms. Seven categories were built in:

Category	Stage	Cause
`bare_int_id`	construct	`id:` unquoted number; UUID() called on an int
`correlation_as_standard_rule`	construct	Correlation syntax parsed as a normal rule
`yaml_scanner_error`	parse	Tabs, bad tokens
`yaml_composer_error`	parse	Value starting with `*`, read as a YAML alias
`yaml_parser_error`	parse	Malformed block structure
`null_date_split`	construct	`date:` present but empty
`null_reference_typeerror`	validate	Null `references:` entry breaks a validator regex

The YAML asterisk trap

Category four catches good engineers. You write a rule detecting a suspicious command line:

```
CommandLine|contains:
    - *cmd /c whoami
```

Looks fine. In YAML, `*` at the start of a value means "alias" — a reference to a named anchor. YAML looks for an anchor called `cmd`, fails, and produces a message unrelated to the actual problem. The fix is quotes, and you'd never guess it from PyYAML's error.

The design decision that paid off

Categories live in two pluggable lists. `PREFLIGHT_CHECKS` examine the raw YAML dictionary *before* pySigma touches it — deterministic, immune to pySigma changing its exception messages. `REGISTRY` matches exception patterns afterwards.

Anything unmatched goes to an `uncategorized_<ExceptionType>` bucket. Nothing is silently swallowed; nothing crashes the run.

Running against real repositories found **four crash classes not in the original seven**:

- `SigmaLogsourceError` (42) — logsource category isn't a string
- `SigmaValueError` (3) — tag missing its namespace prefix
- `SigmaStatusError` (2) — including a rule with the literal typo `'experimental|'`
- `SigmaConditionError` (46, validator stage) — a rule using Sigma's deprecated pipe syntax

That last is the best result in the loader. The rule *parsed successfully*. pySigma accepted it. Then two validators — both internally calling the condition parser without catching its errors — died on it.

Exactly the scenario the two-layer design was built for, confirmed on input we'd never seen. Seven categories became eleven, and adding a named category is a one-line change that doesn't touch the loader.

14. churn.py — reading history without lying

All git access goes through **PyDriller**, an academic library from a 2018 software-engineering paper. Deliberate: when an examiner asks how history was mined, the answer is a citation rather than "I wrote a script."

Following files through renames

This caused a real bug, and catching it validated the approach.

An earlier ad-hoc script tracked files by path string. When a rule moved from `rules/windows/foo.yml` to `rules/windows/process_creation/foo.yml`, that script saw a file that no longer existed and discarded its entire pre-move history.

SigmaHQ has reorganised its directory structure repeatedly. Elastic and Splunk have barely renamed. So SigmaHQ's history was discarded far more, and the numbers came out wrong in a way that changed the ranking between repositories:

Repository	Rename-blind	Rename-aware
SigmaHQ	3.57	7.72
Elastic	6.41	7.38
Splunk	9.42	10.58

SigmaHQ more than doubled and went from apparent outlier to level with Elastic.

The subtle part: *staleness percentages were fine even with the bug.* Staleness needs only the most recent commit, almost always already under the current name. Mean commit count needs full history — exactly what rename-blindness destroys. Same bug, one statistic ruined, the other untouched.

A performance detour

PyDriller computes full text diffs for every file in every commit. mechanic reads only path and change-type metadata. Profiling showed over 80% of an 89-second run generating diffs nobody read. Skipping patch generation dropped it to 40 seconds.

An accidental discovery

Investigating the rename bug revealed that GitPython populates *both* old path and new path with the same value even for a plain file addition. So "old path is empty" does not reliably mean "this file is new."

Verified against real commits rather than assumed. It didn't affect existing output, but it *would* have silently broken creation-commit detection in the semantic diff.

15. ast_repr.py — turning text into shape

A rule is text. Text is hard to reason about. So mechanic converts each rule into an **abstract syntax tree**.

```
          OR
         /  \
      AND    NOT
      /        \
selection    SELECTOR (1 of filter*)
    |          /       \
  leaf    filter_a    filter_b
                          |
                        leaf  (negated: true)
```

Three things this buys.

Knowing what's excluded

Rules contain exclusions: "alert when X happens, *unless* it came from our backup server." A word-counting tool sees "backup server," treats it as something sought, and gets the meaning backwards.

Every node carries a `negated` flag computed from how many NOTs sit above it. A leaf under `not (a or b)` reads `negated: true` regardless of intervening ORs.

Tested against a real SigmaHQ rule combining a top-level OR, a NOT, and a selector. Filter leaves correctly read as negated despite sitting *three structural levels* below the NOT, through two different node types.

Seeing relationships

The renamed-binary rule can only be detected on a tree. In flat text it's just a list of filenames.

Knowing AND from OR

The third turned out to be critical, and nobody realised until Part V. Only a tree tells you whether two conditions are joined by AND or by OR — and that determines how their tiers combine.

Rather than write a parser, mechanic reuses pySigma's own condition parser, requesting the *non*-postprocessed tree so named selections survive as nodes instead of being flattened.

ARMS, a 2026 master's thesis, independently used Sigma-to-AST conversion for rule mutation. Same idea, different purpose, cited as prior art.

16. semantic_diff.py — real change vs cosmetic change

Three methods, in order of preference:

AST comparison (high confidence). Reconstruct the rule before and after the commit, build a tree for each, compare. Ignores key reordering, quoting style and formatting churn.

YAML key comparison (medium). When one side won't load — common in history — compare top-level keys.

Raw text diff (low). When neither side is valid YAML. mechanic reports how often this path was taken.

In practice: 91.9% high confidence on SigmaHQ. Elastic and Splunk fall almost entirely to medium, having no Sigma AST. Low-confidence fallback stayed under 3.2% on four of five repositories.

Two-pass design

Reading diffs requires the patch generation Chapter 14 turned off. So: cheap pass first to find which commits touched which rules, then fetch diffs only for those.

A bug the prediction caught

Before running, a prediction was registered in writing: behavioral staleness must come out *higher* than raw staleness everywhere. Lower anywhere means a bug.

The first run gave joesecurity 29.4% behavioral against 96.6% raw. Impossible.

Root cause: rules with zero behavioral commits were excluded from the calculation, copying a convention from raw staleness where zero organic commits means "no data." But for behavioral staleness, zero is *not* missing data — it means "plenty of history, none of it touched the logic." That's the worst case, not an unknown.

Without the registered prediction, 29.4% looks plausible and ships silently.

17. structural_detectors.py — the idea no wordlist can reach

Four detectors on the tree, each returning a boolean plus the node path that triggered it, so every decision is explainable.

FIELD_MISMATCH

Fires when a rule compares two fields against each other — filename versus internal name, parent versus child process, process name versus signer.

Deliberately narrow: it requires an identity field and a path field asserted under opposite polarity with overlapping basenames, or a direct field-to-field reference between identity fields. A same-user check uses field references too and must *not* trigger it.

The must-pass test: the canonical renamed-binary rule, loaded from the real SigmaHQ clone rather than a fixture, classifies TTP via FIELD_MISMATCH. It passes under every ablation configuration.

ABSENCE

Fires when the rule detects something *not* being there — a missing signature, an empty field, an absent parent.

The hard part is distinguishing this from an exclusion filter. "Alert when the signature field is empty" is absence-based detection. "Alert unless it came from our test server" is noise reduction. Similar in text, opposite in meaning.

RARITY and CORRELATION

Statistical thresholds — counts, frequencies, first-seen, outliers — and logic spanning multiple events.

RARITY needed refinement: it was promoting a rule whose count threshold merely wrapped an ordinary tool-name selector. Now a rarity threshold only promotes when no word in the rule already resolved to Tool tier.

UNSCOREABLE

Some rules contain no literals at all — pure aggregation, machine-learning rules, probability thresholds. Elastic has 106 ML rules with no query by design plus seven with zero literal atoms. These get their own bucket rather than a default tier.

18. protected_literals.py — strings that can't be renamed

Built from the principle in Chapter 7, not from a list of examples. A list assembled from cases you noticed keeps missing cases you didn't.

- Windows autorun registry locations
- Well-known DCE/RPC named pipes
- Well-known security identifiers
- Active Directory schema attribute names
- Linux and macOS persistence paths
- Windows security-subsystem configuration — RDP authentication policy, browser security zones

- Cloud audit action names, context-gated

That second-to-last category was added late, and how illustrates the principle. Four separate classification errors — an NLA-disabled rule, an RDP registry deletion rule, two Internet Explorer security-zone rules — shared one cause: OS-defined registry locations that *weaken security settings*, outside the persistence-scoped autorun category.

Not four bugs. One missing category. Added as a principle; the four errors resolved without any path being typed in by hand. Had they not fallen out of the principle, the principle would have been wrong.

19. fragility.py — combining the pieces

Everything above produces judgements about *parts* of a rule. This decides the rule's overall tier.

For most of the project it did this:

```
rule_tier = max(tier of every positive atom)
```

Simple, intuitive, and — as Part V explains — wrong for the most common case in Sigma. It now walks the AST:

```
AND node       ---> MIN of children   (weakest link)
OR node        ---> MAX of children   (strongest, all must be defeated)
SELECTOR "1 of"---> MAX               (OR semantics)
SELECTOR "all" ---> MIN               (AND semantics)
filters        ---> excluded entirely
negated leaves ---> excluded entirely
```

Excluding filters matches MITRE's own stated rule that filters don't affect robustness — independent corroboration of a design decision mechanic had already made for its own reasons.

Excluding them *entirely*, rather than folding them in at a low value, is critical: a low value inside an AND would drag the minimum down for completely the wrong reason.

One further exclusion is needed. Some fields identify the *data source* rather than describing adversary behaviour — Windows Event IDs, Linux syscall numbers, AWS's `eventSource`, Elastic's `event.provider`, Splunk's `sourcetype`. These aren't independently evadable observables, so including them in an AND's minimum would drag every cloud rule down to the tier of "which service emitted this event."

20. The borrowed parts

A deliberate principle: integrate, don't reinvent.

Need	Used	Instead of
Git history mining	PyDriller (ESEC/FSE 2018)	Parsing `git log` by hand
Rule parsing	pySigma	A custom rule reader
Windows tool names	LOLBAS — 240 binaries	A hand-typed list
Linux tool names	GTFOBins — 478 binaries	A hand-typed list
macOS tool names	LOOBins — 62 binaries	A hand-typed list
Attacker tool names	MITRE ATT&CK STIX — 825 tools	A hand-typed list
Robustness ground truth	MITRE STP scored analytics	Self-generated labels
Running rules on logs	Zircolite (Stage 3)	Building a matching engine
Malicious log samples	EVTX-ATTACK-SAMPLES (Stage 3)	Synthesising logs

2,032 tool names, none typed by hand. Provenance documented for every source — a curated list is precisely what failed in v1.

The limit of outsourcing

Bigger catalogues solve *coverage*. They don't solve *precision*.

GTFOBins legitimately includes `at` and `R`. Real exploitable Unix binaries. They also match ordinary words inside long Splunk queries. The catalogue isn't wrong; the matching was.

The fix wasn't more data. It was **field context**: a tool name counts only when it appears in a field plausibly holding a process, image, command line or parent. `Channel=security` is not a tool reference; `Image=security.exe` would be.

That change moved agreement from 0.747 to 0.800 — and exposed a regression involving Linux auditd's `a0`, `exe` and `comm` fields, a third naming convention that had been missed. Caught before the number was trusted.

Part IV — What We Found

Five repositories, tens of thousands of commits, ninety rules examined individually. Some results confirmed expectations. One replaced the project's headline entirely.

21. Eleven ways a rule repository breaks

Repository	Files	Loaded	Failed	Crash rate
SigmaHQ `rules/`	3,144	3,144	0	0%
joesecurity	119	0	119	100%
mdecrevoisier	353	293	74	21.0%

SigmaHQ loads perfectly — it's the upstream repository the validators were written against, with CI on every pull request.

joesecurity fails completely: 112 bare-integer IDs, 4 YAML scanner errors, 3 composer errors — reproducing the Phase 0 breakdown exactly, confirming the reimplementation was faithful.

mdecrevoisier revealed the four unknown crash classes from Chapter 13. All landed in the fallback bucket. The run finished cleanly.

Windows Defender ate two rule files

During validation, two files vanished from disk. No git operation had touched them. Both were Windows rules with filenames referencing encoded PowerShell.

The working theory: Defender's real-time protection quarantined them, because a Sigma rule detecting encoded-PowerShell command lines *contains those patterns* in order to match them. The detection content is indistinguishable from the threat.

Both hit mechanic's `unreadable_file` category — an I/O failure, not a parse failure, still producing a clean record rather than crashing.

Reported rather than quietly restored, because a rule-triage tool being unable to assume its own input files stay put is a real finding. It is also a small poem about the field.

22. Half of SigmaHQ

Repository	Mean commits/rule	Ever revised	Touched <6mo	Stale >2yr
SigmaHQ	7.72	92.5%	8.1%	51.1%
Elastic	7.38	81.9%	48.0%	2.7%
Splunk	10.58	86.3%	33.8%	5.6%
joesecurity	2.10	33.6%	0.0%	96.6%
mdecrevoisier	4.85	80.6%	0.3%	88.0%

Over half of the world's most widely used public detection rule repository has not had a substantive commit in more than two years.

A detail worth reading twice

SigmaHQ has the *highest* ever-revised rate of all five — 92.5% — and the highest staleness.

So the story isn't "SigmaHQ rules are neglected." It's sharper: **SigmaHQ rules get revised, and then revision stops.** Nine in ten were tuned at least once. Half haven't been touched since 2024.

Rules aren't born abandoned. They're abandoned after a period of care. That is what decay looks like.

23. The real headline: Elastic and Splunk

Repository	Raw stale	Behavioral stale	Change	Never behaviorally touched
SigmaHQ	51.1%	82.5%	+31.4pp	35.3%
Elastic	2.7%	33.2%	+30.5pp	27.4%
Splunk	5.6%	51.9%	+46.3pp	36.8%
joesecurity	96.6%	99.2%	+2.5pp	69.7%
mdecrevoisier	88.0%	93.7%	+5.7pp	35.6%

THE FINDING

By raw commit recency, Elastic (2.7% stale) and Splunk (5.6%) were the two best-maintained corpora in the study, by a wide margin.

By behavioral staleness, **27.4% of Elastic's rules and 36.8% of Splunk's have never had their detection logic revised since the day they were created.**

They were touched often enough — metadata, tags, formatting — to look actively maintained by every measure anyone had.

This beats SigmaHQ's finding for a simple reason. SigmaHQ being stale is believable; it's a volunteer project. Two commercially-backed detection teams looking healthy on every available metric while a third of their logic sits untouched is not believable until measured.

The prediction was directionally right and numerically wrong

The registered prediction said SigmaHQ would rise from 51.1% to "roughly 60–70%." It rose to 82.5% — nearly double the top of the range.

Direction held in all five repositories with zero violations, for the right reason. Magnitude was badly underestimated, reported plainly rather than rounded toward "basically right."

What this cannot yet claim

An obvious objection: *a rule never revised might simply have been correct the first time.*

Fair, and answering it requires crossing behavioral staleness against fragility on the same rules. That experiment is registered and running. Until it returns, this describes maintenance behaviour, not brokenness.

How this was determined

Stated here rather than later: the finding rests almost entirely on medium-confidence key comparison — 99.7% for Elastic, 97.9% for Splunk. Only Sigma gets real AST comparison, at 91.9% high confidence.

That is adequate for the actual question, which is only "did the query field change at all." But it belongs in the same breath as the claim.

24. The kappa climb: 0.412 to 0.800

Cohen's kappa measures agreement between two judgements, corrected for chance. Zero is chance-level, one is perfect. Conventional bands: below 0.4 poor, 0.4–0.6 moderate, 0.6–0.8 substantial, above 0.8 near-perfect.

READ THIS CHAPTER WITH A CAVEAT ATTACHED

Every number here is measured against 90 reference labels that were **generated by a language model**, not a human. Part V and Chapter 32 explain the consequences, which turned out to be serious and instructive.

These figures were also computed under a combination bug corrected later. They are recorded because the sequence matters, not because they represent the current state.

Version	Kappa	What changed
v1	0.412	Word counting only
v2 first pass	0.636	Better lists, negation, structural detectors
+ registry category, CIM fix	0.747	One missing principle, two real bugs
+ field context	0.800	Tool names only count in process fields

What v1 got wrong

38 disagreements of 90, and **36 were under-scoring** — rating good rules as fragile. Systematic, therefore fixable. Five causes:

1. Tool names were Windows-only, missing `grep`, `sudo`, `PlistBuddy`, `python` and everything non-Windows.
2. Protected literals built from examples, not a principle, so they kept missing cases.
3. Event IDs and syscalls excluded as "just data source selectors." Right on average, wrong at the tails — `init_module` *is* the whole rule in a kernel-module detection.
4. Negation blindness — words inside NOT clauses counted as matches.
5. The structural problem, fixed by Chapter 17.

The ablation, and the two flaws in the first attempt

Was the improvement from the structural idea or from bigger wordlists? Turning each off answers it — and the first attempt got it wrong twice.

Flaw one: the "structural only" arm reverted wordlists *further back* than v1's actual baseline, measuring structural detection on a worse foundation than the thing it was compared against. It scored below v1 and appeared to show structure was useless.

Flaw two, bigger: 60 of the 90 test rules are Elastic and Splunk, where structural detection *cannot properly run*. Only Sigma has a real AST. Two-thirds of the test set was structurally blind by construction, and those two-thirds were averaged into the number judging the structural thesis.

Corrected, split by population:

Population	Structural gain, old lists	Structural gain, new lists
Aggregate (n=90)	+0.072	+0.066
Sigma only (n=30)	**+0.092**	**+0.102**
Text only (n=60)	+0.066	+0.051

Structural detection contributes roughly twice as much where a real tree exists. The aggregate figure had been measuring the wrong population.

The number that doesn't reach significance

A bootstrap on the Sigma-only gain gives a 95% interval whose lower bound touches exactly zero, with the effect positive in 86.8% of resamples — roughly $p \approx 0.13$ one-sided. **Not statistically significant at conventional thresholds, at n=30.**

Stated in those words, because rounding it into "we found a significant effect" would be the easiest way to make this project dishonest.

What supports the structural claim isn't the point estimate. It's that the effect is positive, consistently larger on Sigma across *both* independent list configurations, and the canonical renamed-binary case passes under every ablation setting.

The parity nobody should hide

In absolute terms the text corpora scored marginally *higher* than Sigma — 0.801 against 0.796.

Both true at once: structural detection contributes more where a tree exists, and absolute performance was slightly better without one, because the wordlist fixes and the CIM discovery did enormous work on the text side.

25. The CIM discovery

This started as a footnote explaining why Splunk scored worst. It became the largest single lever in the re-validation and a genuine finding about the query language.

The rule that isn't about anything

Splunk has a rule called "Okta New API Token Created." Okta is an identity provider; the rule detects a suspicious API token being made.

The string `okta` appears **nowhere in the rule's search text**.

It exists only in the name of a macro the rule references — and macro resolution consumes that name, expanding the macro into its contents, before anything looks for platform clues. The identical search would fire for any identity provider.

By design, not by accident

Splunk's Common Information Model exists precisely to abstract away vendor differences so one rule works across many products. Removing the vendor's fingerprints is its entire purpose.

The same happens to a Kubernetes rule, where the `kube_audit` macro name is stripped before context detection runs.

> **THE FINDING**
>
> Cloud-context gating requires a visible platform marker in the rule. Splunk's normalisation layer is structurally designed to remove exactly that marker.
>
> A property of the query language, not a defect in the classifier. As far as we can find, nobody has documented it.

The principled fix

The tempting fix is hand-adding every CIM macro name as a marker — exactly the example-driven approach that failed in v1.

The principled fix: look at the macro reference text *before* resolution, using the same existing regex. A widening of *where* the detector looks, not a list of special cases.

Doing this surfaced two real bugs. The regex used word boundaries, which could never match Splunk's underscore-compounded macro naming. And Kubernetes' `verb` field — that platform's documented audit action-name field — was missing entirely.

Splunk's agreement moved from 0.505 to 0.703. Worth being clear: that was a bug fix, correctly found and honestly reported. Not evidence for the structural thesis.

Part V — The Turning Point

Everything up to here was measured against labels the project generated itself. This part is what happened when it was finally measured against something external — and why the resulting collapse was the best thing that happened to it.

26. Looking for a ruler

There was a problem sitting under the whole of Part IV, and it needs stating without softening.

The 90 reference labels — against which 0.412, 0.636, 0.747 and 0.800 were all measured — were **not produced by a human annotator**. They were generated by a language model during Phase 0, stored in a file named `phase0_hand_labels.csv` with a column called `hand_tier`, and treated as ground truth thereafter.

Three roles that should belong to different parties belonged to one: the same agent helped define the taxonomy, produced the labels, and reviewed the classifier measured against them.

And the failure mode is invisible in principle. If the labeller has a systematic bias, a classifier tuned toward those labels inherits it — and kappa goes *up*. The number improves as the shared error worsens. Agreement cannot detect correlated mistakes.

So: find an external ruler. Two candidates.

Candidate one: SigmaHQ's own tags

The Sigma specification defines an `stp` tag carrying a MITRE robustness score. If enough rules carried one, that would be human-assigned ground truth for the same construct, already on disk.

Result: 6 tags out of 3,783 files. One commit, one author, methodology v1.0.0. Chapter 4 tells that story. Statistically useless as a validation set — and, as it turned out, a finding in its own right.

Candidate two: MITRE's own scored analytics

The STP project publishes a CSV of analytics scored by MITRE's researchers directly, most carrying a permalink to the exact commit of the source rule plus a free-text justification.

90 rows parsed. **78 resolved to a real rule file at the exact scored commit** and were classified by mechanic. The 12 exclusions were each checked and stated: four had no permalink at all; three carried a literally ambiguous score in the source CSV, MITRE's own researchers marking uncertainty; two pointed at paths that no longer exist; one pointed at a third-party repository not cloned.

Externally produced, human-expert, same construct, with commit-pinned provenance. This became the primary validation route.

Mapping the scales, before looking at any result

STP scores analytic robustness 1–5. mechanic has four tiers. A mapping was written and documented *before* any agreement figure was computed — because deciding a mapping after seeing results is one of the easiest ways to fool yourself.

Two candidate mappings were tested for sensitivity, differing in where STP level 3 lands. And the **headline statistic is rank correlation on the unmapped scales**, precisely so the mapping choice isn't load-bearing.

27. The bug that eight rounds missed

Reading STP's methodology carefully — still before computing any correlation — turned up a sentence that changed the project.

> *AND combines by minimum. OR combines by maximum.*

An AND-linked analytic is only as robust as its weakest linked observable, because the adversary need only defeat one. An OR-linked analytic is at least as robust as its strongest, because the adversary must defeat all.

mechanic's `rule_tier` was `max()` over every positive atom, **regardless of AND/OR structure**.

Correct for OR-linked rules. Wrong for AND-linked ones. And a Sigma `selection:` block combines its fields by implicit AND — so the wrong case was the common case.

This had been there since v1. It survived Tasks 1 through 8. It survived every kappa improvement, every ablation, every disagreement review.

What the numbers said

First, mechanic as actually shipped, against MITRE's scores:

Sample	n	Kendall's tau-b	p
Primary	72	0.032	0.767
Including provisional rows	78	-0.050	0.630

Statistically indistinguishable from zero. Per the pre-registered interpretation, this was the "no correlation" branch — reported as the primary finding rather than buried.

Then the same 70 Sigma rules, scored two ways:

Scoring method	tau-b	p
mechanic as shipped (`max()` over all atoms)	-0.009	0.934
STP-aware (AND → MIN, OR → MAX)	**+0.300**	**0.0082**

THE DIFFERENCE BETWEEN NOTHING AND SOMETHING

Same 70 rules. Same classifier. Same lists, same detectors, same protected literals. The only change is which operator combines AND-linked atoms.

22 of the 70 rules change tier, and the pattern is completely one-directional: **every single change moves down**. The old logic was systematically over-scoring.

Two examples make the reasoning concrete.

Windows Binaries Write Suspicious Extensions, which MITRE scored at the weakest level with the justification that it depends on target filenames that can easily be changed. mechanic's `max()` found a higher-tier atom elsewhere in the same AND-linked selection and reported Tool. The corrected MIN reports Artifact — because the adversary only has to evade the weakest AND-linked part, so the filename-dependent condition determines robustness, not whatever strongest atom happens to sit beside it.

Direct Syscall of NtOpenProcess: `max()` found a syscall-tier atom and reported TTP. MIN reports Artifact, far closer to MITRE's actual score.

After the fix

	n	Kendall's tau-b	Spearman's rho
Before	72	0.032 (p=0.767)	0.035
After	72	**0.361 (p=0.0010)**	**0.392 (p=0.0007)**

Weighted kappa moved from 0.074 to 0.316 alongside it.

A moderate, highly significant positive correlation against analytics scored by MITRE's own experts. The pre-registered interpretation called this the strongest available validation branch.

A second fix, needed within minutes

The very first production re-run showed SigmaHQ's kappa against the development set collapsing to 0.188, with 19 of 30 rules flipping — almost all cloud-audit rules.

Traced before being accepted: AWS CloudTrail's `eventSource` field (values like `ec2.amazonaws.com`) was AND-linked

with the substantive, protected `eventName` field. The new MIN logic correctly-by-the-letter dragged the whole rule down to `eventSource`'s generic classification.

But `eventSource` is a *data-source identifier* — which AWS service emitted the event — not an independently evadable observable. Exactly the category Event IDs and syscalls were already excluded from, for exactly the same reason.

Generalised on that existing justification rather than to fix one rule: `eventSource`, Elastic's `event.provider`, ECS's `data_stream.dataset`, Splunk's `sourcetype`. A regression test locks it in.

Worth noting honestly: this fix was *discovered by watching the development-set kappa fall*, which sits on the wrong side of the project's own rule against tuning to the test set. The justification is principled and the generalisation wasn't fitted to one case — but the discovery path ran through the dev set, and that should be said rather than glossed.

28. The number that fell, and why that was good news

The fix improved external validation dramatically. It also did this:

	Aggregate	Sigma-only	Elastic	Splunk
Before both fixes	0.800	0.796	0.780	0.798
After	**0.615**	**0.298**	0.780	0.798

SigmaHQ collapsed from 0.796 to 0.298. The 95% bootstrap interval [0.132, 0.485] doesn't overlap the old estimate at all. Disagreement direction flipped hard: 23 under-scores against 1 over-score.

Elastic and Splunk are *exactly unchanged*, because their scoring path has no AST to apply AND/OR structure to. More on that in a moment.

What caused the fall

Investigated rather than accepted. One example: a rule detecting a Linux setgid capability being set. `Image|endswith: '/setcap'` classifies Tool, correctly. AND-linked with `CommandLine|contains: 'cap_setgid'`, which mechanic's lists classify as a generic Artifact literal. Under MIN the rule now scores Artifact overall.

But is `cap_setgid` actually a weak, freely renamable link? Arguably not — it is a POSIX capability name defined by the Linux kernel's own interface, not an adversary-chosen string. Which means it should be a *protected literal* that mechanic doesn't currently recognise.

> **THE MECHANISM, STATED DIRECTLY**
>
> The old flat-max logic was **masking** gaps in atom classification. When any one atom in an AND-linked group scored highly, the whole rule inherited that score — regardless of how badly the other atoms were classified.
>
> So kappa looked better than the classifier's true atom-level precision. Removing the masking is more correct by the external measure, and simultaneously reveals that atom coverage has more gaps than were previously visible.

And here the project applied its own standing rule. Those newly-exposed gaps were **not** chased and patched against the 90-rule set. Fixing individual atoms that this specific investigation happened to surface is precisely the whack-a-mole the instruction exists to prevent. They are logged as known, disclosed, not-yet-addressed work for a future round using a fresh held-out sample.

Which number is the result now

The **STP correlation** — external, human-expert, statistically significant — is the number that validates the taxonomy and the classifier's core logic.

The development-set kappa is an internal, self-generated figure now shown to have been partly inflated by a masking artifact in buggy combination logic. The earlier 0.747 and 0.800 should be read as "kappa under the since-corrected combination bug," not as the classifier's current state.

29. What the collapse proved about LLM labels

There is a second reading of Chapter 28 that deserves stating on its own, because it is a finding about method rather than about detection rules.

The LLM-generated labels **agreed with the bug**.

Correcting mechanic toward MITRE's published methodology moved it *away* from the LLM labels. Agreement with the external human standard rose from nothing to significant; agreement with the internal LLM labels fell by half.

That is direct empirical evidence of correlated error. The labeller and the classifier shared a systematic mistake — both reasoning that a rule is as strong as its strongest component — and agreement between them was *elevated* by that shared mistake.

> **THE METHODOLOGICAL FINDING**
>
> When ground-truth labels are generated by a language model and the system being evaluated reasons similarly, agreement measures shared intuition rather than correctness. The metric improves as the shared error deepens, and no amount of internal validation can detect it.
>
> Eight rounds of fixes measured against LLM labels never surfaced the combination bug. One afternoon against MITRE's data did.

Both explanations of the collapse are true simultaneously — some is genuine classifier weakness now exposed, some is LLM bias no longer being flattered. Separating them requires the human-labelled sample described in Chapter 39.

This is worth publishing on its own. It is a concrete, quantified demonstration of a failure mode that is widely suspected and rarely measured, in a domain where LLM-assisted evaluation is becoming common.

Part VI — Being Wrong Properly

The most important part of the book. A project reporting only what worked is marketing. These mistakes are recorded because catching them is what makes the rest trustworthy.

30. The retraction

An early round produced a striking result: **only 3% of detection rules across every corpus reach TTP tier.** Nobody writes durable detection. Amateurs and professionals alike, 3% either way.

A great finding. Completely wrong.

The cause was an over-strict protected-literal definition. The classifier matched field names exactly instead of by last dotted segment, so `event.action` failed to match a rule for `action`. Fields that should have been recognised as protected weren't.

Fixing that one thing moved Elastic's automated TTP share from 3.8% to 24.8% — a 6.5-fold swing from a single line. Careful re-labelling of the same SigmaHQ corpus gave roughly 36.7%.

Caught for one reason: **someone read a sample of the output by hand before trusting the number.** The aggregate looked plausible. The individual rules didn't.

> **STANDING INSTRUCTION**
>
> The 3% figure must never appear anywhere, and the retraction must be stated explicitly in any write-up.
>
> A paper saying "our first method gave 3%, here is exactly why that was wrong" is more credible than one that never made the mistake.

31. The citation that never existed

A quotation was used repeatedly across the project — "green CI is a dangerous green," attributed to a named author — to support the point that CI validates syntax but not semantics.

When it was finally checked, no paper, talk or post by that name makes that claim. The attribution was fabricated somewhere in the reasoning chain and then repeated as though solid, surviving several drafts.

It was removed rather than quietly substituted, and the gap recorded. The underlying observation is sound and the project's own data demonstrates it directly — so it stands as a finding, not a borrowed quote.

This is why Appendix D exists, and why every figure in this document should be traceable to a source with a date. The failure mode is not malice; it is a confident-sounding sentence that nobody thought to check because it fitted so well.

32. Everything else we got wrong

A list, because the pattern matters more than any item.

The core combination operator, for the entire project

`rule_tier = max(atoms)`, wrong for AND-linked rules, which are the common case in Sigma. Present since v1, survived eight rounds of improvement, invisible to every internal metric, found only by reading an external methodology. Chapter 27.

Worth sitting with: this was not a subtle edge case. It was the central calculation, and the project's own validation could not see it.

Treating LLM output as ground truth

Ninety labels generated by a language model, stored under a filename saying "hand labels," and used as the reference for every agreement figure across four rounds. The naming made it look like something it wasn't, and

nobody challenged it for months. Chapters 26 and 29.

The thesis was too big

The original claim: *durability is a property of query structure, not of the literals a query matches.*

The ablation showed wordlist fixes recovered about 68% of the improvement. Structure contributed real but smaller aggregate lift. The claim was cut to what the evidence supports.

An unfair experiment, twice over

The first ablation reverted wordlists further back than the true baseline, and averaged in 60 rules where structural detection cannot run. Both flaws pushed toward the same wrong conclusion. Chapter 24.

A fix that did nothing

Over-scoring was diagnosed as short tool names like `at` and `R` colliding with ordinary words. A suppression rule for names under three characters was implemented and tested.

It had **zero measured effect**. The real collisions were longer words — `security`, `replace`, `url`. Reported as a still-open gap rather than claimed as fixed. The actual fix was field context.

A prediction underestimated by double

SigmaHQ's behavioral staleness predicted at 60–70%. It came out at 82.5%. Direction right, magnitude badly wrong, reported as such.

Refusing to round

The pre-registered threshold for "substantial agreement" was 0.65. The first v2 result was 0.636. Everything invited rounding it up. It was reported as below threshold.

Later fixes took it past 0.65 legitimately. Having not claimed it early is what makes claiming it later mean anything — and, as Part V showed, that number was inflated anyway.

One fix on the wrong side of a line

The `eventSource` exclusion was discovered by watching development-set kappa collapse. The justification is principled and the generalisation wasn't fitted to one case — but the discovery path ran through the test set, against the project's own rule. Stated rather than glossed. The subsequent decision *not* to chase POSIX capability names and other newly-exposed gaps drew the line correctly.

33. Limits, stated plainly

What mechanic can and cannot say

Can say: "This rule keys on something an attacker can change cheaply." That holds regardless of environment; it's a property of the rule.

Cannot say: "This rule will fail in your environment." That needs real logs from that environment.

The text-language gap just got worse

This was a known limitation. Part V made it serious.

Elastic and Splunk are scored by a path with no AST. So they are still using `max()` over atoms — the combination operator external validation showed to be wrong — with no structural information available to fix it. Their kappa was *exactly unchanged* by the correction, which is precisely the problem.

Before Part V, text-language support was "weaker, capped at medium confidence." Now it is "using a demonstrably incorrect combination operator, unfixable without real parsers for EQL, KQL and SPL."

That must appear on the tool's output for those corpora, not only in documentation.

Validation covers one language

The STP scored-analytics set is essentially all Sigma. The correlation validates the Sigma path only. Nothing external has yet tested the text-language classifier.

Organisations modify their rules

Companies adapt public rules, so the exact rules analysed aren't the ones running in production.

This matters less than it appears. The property measured — what a rule keys on — travels with the rule when copied. And the commonest local modification is *adding exclusions*, each making a rule slightly more fragile. Local copies are likely worse than baseline, not better.

Sample sizes

78 rules resolved for STP validation, 72 in the primary sample. Ninety for the development set, thirty for Sigma-only analysis. All small. Bootstrap intervals reflect this honestly.

The claim about STP adoption

Absence of published tags is not proof of absence of use. Enterprises may score internally and never publish. The defensible claim is about *public, shared* scoring — which is what MITRE asked for. Phrased that way it holds; phrased more broadly it doesn't.

Part VII — Where It Stands

34. Current status, honestly

Component	State
Fault-isolated loader	Complete, validated on five repositories
Staleness engine	Complete, reproduces prior figures
Rule AST	Complete, Sigma only
Semantic diff	Complete, validated on five repositories
Fragility classifier	Complete for Sigma; text path uses known-wrong combination logic
External validation	Complete for Sigma — tau = 0.361, p = 0.0010
Priority ranking (Part 3)	**Not built** — blocked on the correlation experiment
Repair engine (Stage 3)	Designed, not built
Verification harness (Stage 3)	Designed; Zircolite identified, not integrated
Human-labelled validation set	Not started
Text-language parsers	Not started

The honest one-paragraph summary

mechanic can open rule repositories that crash the standard tooling, measure how long since anyone meaningfully touched each rule, and score Sigma rules for evasion resistance in a way that correlates significantly with MITRE's own expert assessments. It cannot yet rank rules by priority, cannot repair anything, and its scoring for Elastic and Splunk uses combination logic now known to be incorrect.

35. What is running right now

Two experiments are in flight, and both are blocking.

The correlation experiment

Does behavioral staleness predict fragility? It re-mines full git history for three large repositories on top of everything already computed, so it takes hours.

The prediction is registered and will not be adjusted:

> *Never-behaviorally-revised rules will skew toward the fragile tiers relative to revised rules. If there is no association, or it runs the other way, the premise that these two signals should be combined is unsupported and the ranking design must change.*

Design: contingency table with chi-square and Cramér's V; a 2×2 collapse with Fisher's exact for small cells; Spearman correlation against days-since-behavioral-change to catch a gradient a binary split would hide; run on both the full corpora and the development set; and an age control, because older rules have had more opportunity to be revised.

Three possible outcomes, interpretation fixed in advance:

- **Association survives age control** — the signals validate each other and the combined ranking rests on evidence
- **No association** — not fatal; both stay individually useful, but the ranking must present them as independent axes and say why
- **Association vanishes under age control** — age is doing the work, not maintenance behaviour, and that is the finding

The STP adoption census

Extending the six-tag SigmaHQ scan across every Sigma-format repository obtainable, with exact provenance for every figure: release dates from the project's own changelog, the scored-analytics CSV's exact filename and row count, the precise wording and location of MITRE's request for community contributions, and elapsed time stated as a precise interval rather than "three years later."

The argument is strong. Loose wording would hand a reviewer an easy target.

36. The contribution, in its final form

The framing has narrowed three times, each time under pressure from evidence, and each time it got more defensible.

First version: "A tool that finds fragile detection rules." Too broad — sigmalint scores quality, Uetz proved fragility, the space was crowded.

Second version: "The first tool that repairs detection rules." Wrong — GRIDAI and EvoSIEM got there, reactively.

Third version: "Corpus-wide prioritisation of which rules need attention." Better, and survived a targeted search.

Current version, after STP:

> *Summiting the Pyramid defines how to score detection-analytic robustness, is endorsed by MITRE and six industry partners, and is formally adopted into the Sigma specification — yet public application stands at 6 tagged rules out of 3,783, because scoring requires a human expert per analytic. mechanic automates STP-style scoring at corpus scale, validated against MITRE's own expert-scored analytics, and combines it with behavioral maintenance history to prioritise which existing rules deserve an engineer's attention.*

Why this is the strongest version:

- It doesn't claim to invent a taxonomy — MITRE did, and is cited
- The gap is measured, not asserted — 6 out of 3,783
- Validation is external and significant — tau = 0.361, p = 0.0010
- It states what remains manual, and why automation is the missing piece
- It is falsifiable — and was very nearly falsified, before the AND/OR fix

There is a secondary contribution worth stating separately, because it is independent of whether mechanic ever ships: a quantified demonstration that LLM-generated evaluation labels can share a systematic error with the system being evaluated, inflating agreement while hiding a central bug — detectable only by external validation.

Part VIII — What Comes Next

37. Part 3: the priority ranking

This is the contribution. Everything built so far exists to make it possible.

It takes, per rule: behavioral staleness, fragility tier, classification confidence, the unscoreable flag, and which structural detectors fired. It produces an ordered list with reasons.

Six requirements

Explainability is mandatory. Every score lists its contributing signals with values. A ranked list without reasons is a black box, and no engineer will act on one.

"Review this," never "this is bad." tau = 0.361 is real, significant, and nowhere near authoritative.

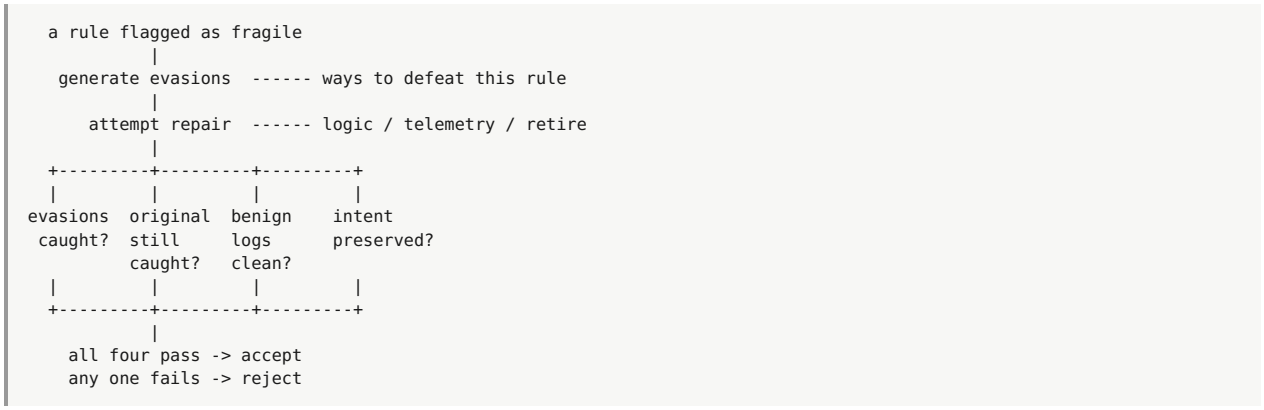
Confidence must propagate. Elastic and Splunk rules carry the caveat that their tiers use the known-wrong combination operator — on the output, not in a footnote.

Unscoreable rules get their own section. Never a default tier, never silently dropped.

Weighting configurable, default justified. Same discipline as the 10% threshold, with ranking stability reported under alternative weightings.

Triage hypotheses labelled as hypotheses. Likely-repairable, likely-needs-telemetry, likely-retire — for Stage 3 to test, not conclusions.

38. Stage 3: repair and proof



Running without a SIEM

Zircolite, built on pySigma, runs Sigma rules against Windows event logs and JSON using a SQLite backend. **EVTX-ATTACK-SAMPLES** provides public logs containing real attack activity.

Together, the verification loop runs on a laptop. That component was expected to consume a month. It's mostly a wrapper around existing tools.

Where AI belongs, and where it doesn't

Almost everything in mechanic is deterministic. Same repository, same answer, every decision traceable.

A language model has one job in Stage 3: **suggesting** repaired logic. Its output is a proposal, never a conclusion. The verification loop decides truth.

Part V made this argument stronger than it was. A model's plausible-sounding reasoning agreed with a bug in the core logic for months, and no internal metric caught it. Anything a model produces here must be checkable against something outside the model.

Independent evasions

Evasions written by the same system that writes repairs are not independent evidence — the same reasoning marking its own homework. Exactly the failure Part V documented.

The Fraunhofer artifact contains 512 expert-crafted evasions, withheld from public release for ethical reasons but available to researchers on request. Obtaining them would remove the largest source of uncertainty in Stage 3.

39. Open questions

Does staleness predict fragility?

Running. Determines whether Part 3 produces a combined score or independent axes.

How wrong is the text-language path?

The most urgent question. Elastic and Splunk use combination logic that external validation showed to be incorrect for Sigma. There is no reason to expect it correct for them.

Answering it needs real parsers for EQL, KQL and SPL. Substantial work, and now clearly the highest-value engineering task remaining.

What would human labels show?

Thirty rules, written protocol, blind labelling, ideally a second labeller. Three numbers: classifier-vs-human, LLM-vs-human, human-vs-human.

The third establishes the **ceiling**. If two experts agree at 0.70, a classifier at 0.75 is performing above human consistency — a far stronger result than "falls short of ground truth."

It would also separate the two explanations of the kappa collapse: genuine classifier weakness, or LLM bias no longer flattered.

How many atom-classification gaps remain?

The AND/OR fix removed a masking effect and exposed gaps that were always there. POSIX capability names are one identified example; almost certainly not the only one.

Deliberately not chased against the existing test set. Needs a fresh held-out sample.

What about rules nobody can score?

Machine-learning rules and pure statistical thresholds have no literals. They sit in an unscoreable bucket. Whether they are durable, fragile, or a different kind of thing is unknown.

Will MITRE automate this first?

STP v4.0's changelog describes automating part of their scoring using AI and LLM assistance — on telemetry confidence, a different axis. But they have the domain expertise, the ground truth and the funding.

An argument for writing up sooner rather than later. Also, honestly, validation: the organisation that invented the methodology has concluded manual application doesn't scale.

Appendices

A. Glossary

Term	Meaning
Ablation	Turning a component off to measure its contribution.
AST	Abstract syntax tree. A rule as a structure of relationships instead of text.
ATT&CK	MITRE's catalogue of attacker techniques, each with an ID like T1003.
Behavioral staleness	Time since anyone changed a rule's detection logic, ignoring cosmetic edits.
CIM	Splunk's Common Information Model. Normalises across vendors — and strips vendor identity doing it.
Cohen's kappa	Agreement between two judgements, corrected for chance.
Detection rule	An if-statement over log data that raises an alert when it matches.
EQL / KQL / SPL	Query languages. First two Elastic's, third Splunk's.
Evasion	A modified attack achieving the same goal without triggering a rule.
Fragility	How cheaply an attacker can change what a rule depends on.
IOC	Indicator of compromise. A hash, IP or domain. Most fragile.
Kendall's tau-b	Rank correlation between two ordered scales. Used because it needs no scale mapping.
Logsource	The part of a Sigma rule saying which logs it applies to.
Mechanical commit	A bulk change touching a large fraction of a repository. Not maintenance.
Organic commit	What remains after mechanical commits are excluded.
OriginalFileName	Identity compiled into a Windows executable. Renaming doesn't change it.
Protected literal	A fixed string the attacker cannot rename, because a platform defines it.
Pyramid of Pain	Bianco's model ranking detection targets by cost to the attacker.
pySigma	The standard Python library for parsing and validating Sigma rules.
Sigma	An open, vendor-neutral YAML format for detection rules.
STP	Summiting the Pyramid. MITRE's methodology for scoring analytic robustness.
Telemetry	The log data available to detect with.
TTP	Tactics, techniques and procedures. Behaviour rather than tooling.

B. Prior art

Work	What it does	What it leaves undone
Summiting the Pyramid (MITRE CTID)	Formal robustness scoring methodology, v4.0, six industry partners, adopted into the Sigma spec	Requires manual expert scoring per analytic; 6 public tags in SigmaHQ
pySigma validators	80 checks on rule form and hygiene	Nothing about evasion
Uetz et al. (USENIX Sec 2024)	292 rules hand-analysed; 38% fully evadable, 7% partially; AMIDES detects evasions	Manual, Windows process creation only; detects rather than prevents
sigmalint (2026)	Six-dimension quality scoring	Fragility not among them; SigmaHQ only
GRIDAI	Multi-agent LLM generation and repair with hallucination validation	Reactive — triggered by incoming attack samples
EvoSIEM (Dec 2025)	LLM hardening of Sigma rules against evasion	Operates on rules handed to it
ARMS (2026 thesis)	Generates rule variants for related techniques via three-dimensional proximity	Coverage expansion; never asks if the seed rule is good
RuleGenie	Rule-set optimisation via embeddings and similarity	Redundancy focus, not durability
AutoSigma, LLMCloudHunter, IntelEX, SigmaGen, RulePilot, FALCON	Generate new rules from threat intelligence	Nothing about existing rules
CardinalOps, Cymulate, Spectrum Security	Commercial detection posture and coverage	Closed, enterprise-priced, coverage rather than fragility

C. Every number in one place

Loading

- `sigma check` crashes on 4 of 5 real third-party repositories
- joesecurity: 119 files, 0 rules loaded, 100% failure
- 7 crash categories designed; 11 found in practice
- SigmaHQ: 3,144 files, zero load failures, zero validator crashes

Staleness

- Largest single commits: SigmaHQ 2,931 files; Splunk 2,068; Elastic 1,064
- Raw stale >2yr: SigmaHQ 51.1%, Splunk 5.6%, Elastic 2.7%
- Behavioral stale >2yr: SigmaHQ 82.5%, Splunk 51.9%, Elastic 33.2%
- Never behaviorally revised: Splunk 36.8%, SigmaHQ 35.3%, Elastic 27.4%
- Rename-blindness correction: SigmaHQ mean commits/rule 3.57 → 7.72
- High-confidence AST comparison: SigmaHQ 91.9%; Elastic and Splunk almost entirely medium

Classification, internal (development set, LLM-labelled)

- v1: 0.412, 57.8% agreement, 36 of 38 disagreements under-scoring
- Peak before AND/OR fix: 0.800 aggregate, 0.796 Sigma-only
- After AND/OR fix: 0.615 aggregate, 0.298 Sigma-only
- Structural marginal gain: +0.092 to +0.102 Sigma; +0.051 to +0.066 text
- Bootstrap lower bound touches zero; $p \approx 0.13$; not significant at n=30
- Splunk 0.505 → 0.703 from the CIM fix alone
- Tool vocabulary: 2,032 names from four external catalogues

Classification, external (MITRE STP — the result that counts)

- 90 CSV rows; 78 resolved to real rules; 72 in the primary sample

- Before AND/OR fix: tau-b = 0.032, p = 0.767 — indistinguishable from zero
- **After: tau-b = 0.361, p = 0.0010; Spearman 0.392, p = 0.0007**
- Weighted kappa 0.074 → 0.316
- Diagnostic on 70 Sigma rules: max() gave -0.009; AND→MIN gave +0.300, p = 0.0082
- 22 of 70 rules changed tier; every change moved down
- Event-robustness correlation: 0.010, p = 0.944 — mechanic has no equivalent axis

STP adoption

- SigmaHQ: 6 `stp` tags across 3,783 files (0.16%)
- All 6 in one commit, one author, methodology v1.0.0
- MITRE's own scored corpus: approximately 90 analytics

Retracted

- "3% of rules are TTP tier" — wrong. Corrected figure roughly 36.7%.
- "Green CI is a dangerous green" — unverifiable attribution, removed.
- Kappa 0.747 and 0.800 — measured under a since-corrected combination bug.

External evidence

- Uetz et al.: 292 rules, 38% fully evadable, 7% partially, 512 hand-crafted evasions
- SCYTHE: 57 rules regression-tested, 87.7% pass rate — 1 in 8 silently failed
- Spectrum Security raised $19M for automated detection maintenance

D. Datasets and sources

Every external dependency, so a reader can reproduce the work. **Every entry must be pinned to a commit hash or release tag with a retrieval date before publication** — kappa figures are not reproducible against a moving catalogue.

Rule corpora

Name	URL	Used for
SigmaHQ/sigma	github.com/SigmaHQ/sigma	Primary corpus; loading, staleness, AST, classification
elastic/detection-rules	github.com/elastic/detection-rules	Staleness, text-path classification
splunk/security_content	github.com/splunk/security_content	Staleness, text-path classification
joesecurity/sigma-rules	github.com/joesecurity/sigma-rules	Loader stress test — 100% crash rate
mdecrevoisier/SIGMA-detection-rules	github.com/mdecrevoisier/SIGMA-detection-rules	Loader stress test; four new crash classes
Yamato-Security/hayabusa-rules	github.com/Yamato-Security/hayabusa-rules	STP tag census (pending)

Ground truth and validation

Name	URL	Used for
Summiting the Pyramid	github.com/center-for-threat-informed-defense/summiting-the-pyramid	**Primary external ground truth.** Scored-analytics CSV; methodology; combination rules
STP documentation	center-for-threat-informed-defense.github.io/summiting-the-pyramid	Scale definitions, changelog, release dates
Sigma specification	github.com/SigmaHQ/sigma-specification	The `stp` tag definition
AMIDES (Uetz et al.)	github.com/fkie-cad/amides	Evadability labels; 512 evasions (full set on request)

Tool and binary catalogues

Name	URL	Contributes
LOLBAS	github.com/LOLBAS-Project/LOLBAS	240 Windows binaries
GTFOBins	github.com/GTFOBins/GTFOBins.github.io	478 Linux/Unix binaries
LOOBins	github.com/infosecB/LOOBins	62 macOS binaries
ATT&CK STIX data	github.com/mitre-attack/attack-stix-data	825 named attacker tools

Libraries and Stage 3 dependencies

Name	URL	Used for
pySigma	github.com/SigmaHQ/pySigma	Rule parsing, validators, condition parser
PyDriller	github.com/ishepard/pydriller	All git history mining (ESEC/FSE 2018)
Zircolite	github.com/wagga40/Zircolite	Stage 3 — running rules against logs offline
EVTX-ATTACK-SAMPLES	github.com/sbousseaden/EVTX-ATTACK-SAMPLES	Stage 3 — malicious log corpus

E. Timeline

Phase	What happened	Outcome
Phase 0, round 1	Surveyed pySigma validators, ATT&CK handling, SigmaHQ cleanliness	80 validators exist; none address fragility
Phase 0, round 2	Tested third-party repositories	4 of 5 crash; seven root causes; two-layer isolation required
Phase 0, round 3	Professional corpora; hand classification	The 3% retraction; protected literals; the structural insight
Stage 1	Loader, staleness engine, AST	Eleven crash classes; 51.1% SigmaHQ staleness; rename bug found
Stage 2, part 1	Semantic diff	The Elastic/Splunk behavioral finding
Stage 2, part 2	Classifier v2, eight rounds of fixes	Kappa 0.412 → 0.800 against LLM labels
Stage 2, external	**MITRE STP validation**	**AND/OR bug found; tau 0.032 → 0.361**
Stage 2, aftermath	Re-ran everything	LLM kappa fell to 0.615; masking effect identified
Now	Correlation experiment; STP census	Running
Next	Part 3 priority ranking	Blocked on correlation result
Then	Stage 3 repair and verification	Designed

Afterword: why the mistakes are in the book

A reasonable person might ask why a document explaining a tool spends an entire part on things that went wrong, and devotes its longest chapter to a bug in its own core logic.

Because in this project, the mistakes *are* the method.

The 3% finding was caught by reading output by hand instead of trusting an aggregate. The behavioral staleness bug was caught by a prediction written down before the run. The unfair ablation was caught by asking what population the number was measured on. The fabricated citation was caught by trying to verify it. The ground-truth problem was caught by asking who, precisely, produced the labels.

And the biggest one — the combination operator that had been wrong since the first line of the classifier — was caught by none of those. It was caught by measuring against something the project did not produce itself.

That is the lesson worth carrying out of here. Internal consistency is not correctness. A system can be self-consistent, well-tested, carefully ablated, honestly reported, and wrong in the same direction throughout — especially when the thing it is measured against shares its assumptions.

Eight rounds of improvement measured against labels the project generated made the number go up while the central calculation stayed wrong. One afternoon measured against MITRE's data made the number go down and the tool correct.

The thing being built tells security engineers which of their rules they can no longer trust. It would be a strange kind of failure to build that on numbers nobody outside the project had ever checked.

> *Detection rules quietly stop working, and nobody notices. mechanic finds the ones that are stale or easy to dodge, will repair them where it can, and proves the repair worked — while being honest about the parts it cannot yet do, and the parts it once got wrong.*

This is a living document. Sections marked as running or blocked
will be updated as results arrive.